HW5 — Quality Software Documentation Report

CSC 640 — Software Quality

Author: Nadine Karabaranga

Instructor: Dr. Samuel Cho

Week 1 — Foundations of Quality Software

1. Introduction

Week 1 introduced the foundational concepts of quality software engineering. The lectures emphasized that building high-quality software requires more than functionality; it requires reliability, maintainability, predictability, and clear communication. These principles form the basis for professional software development.

2. Understanding Quality Software

Quality software includes the following characteristics:

- **Reliable** — performs correctly under expected and unexpected conditions
- **Maintainable** — easy to update, extend, and debug
- **Predictable** — behavior remains consistent across versions
- **Adaptable** — built to evolve with future requirements

Building maintainable software is ultimately building for the future.

3. Managing Complexity

Professional software engineers manage complexity using:

- Modular architecture
- Clear naming conventions
- Abstraction and encapsulation
- Continuous testing
- Thorough documentation

Effective complexity management reduces errors, improves clarity, and enhances performance.

4. The “No Surprises” Principle

The *No Surprises* rule encourages transparent collaboration:

- Communicate progress and blockers early
- Avoid unexpected code changes
- Notify teammates of risks
- Maintain a predictable workflow

This principle strengthens trust and maintains alignment.

5. The ASE Framework

The ASE 4-D Cycle includes:

1. **Define** — clarify objectives and tasks
2. **Design** — outline system structure
3. **Develop** — implement and test code
4. **Deploy** — release and evaluate

This framework enhances traceability and quality.

6. Tools Supporting Week 1

Category	Tools	Purpose
Version Control	GitHub	Tracks changes and enforces transparency
IDE	VS Code	Unified development environment
Testing	Postman, PHPUnit	Validate correctness
Documentation	Markdown, Marp	Communicate design and results

7. Reflection

Week 1 reinforced that software quality relies on communication, structure, and consistency. The tools and principles introduced became essential for HW4 and subsequent weeks.

8. Conclusion

High-quality software emerges from strong foundations—manageable complexity, clear planning, and collaboration. Week 1 set the standard for all following coursework.

Week 2 — Software Process and Development Models

1. Introduction

Week 2 examined why a structured software process is crucial for building predictable and maintainable systems. A formal process guides design, implementation, and evaluation across the development lifecycle.

2. Importance of Process

A clear process provides:

- Predictability
- Reduced risk
- Shared understanding
- Accountability
- Repeatable success

Without process, projects drift and become unstable.

3. Classical Software Development Models

Waterfall

- Linear, phase-oriented
- Suitable for stable requirements

Spiral

- Iterative with risk assessment
- Ideal for large, complex problems

Agile/Scrum

- Short development cycles (sprints)
- Strong emphasis on adaptability and feedback

4. Tools Supporting Process

- Version control via GitHub
- Issue tracking and milestones
- Documentation standards
- Automated testing tools

These reinforce discipline and structure.

5. HW4 Application

HW4 followed a structured development process:

1. Database schema planning
2. Controller and route implementation
3. Authentication via Bearer tokens
4. Postman test cycles
5. Deployment via NGINX

This ensured controlled progress and steady improvement.

6. Reflection

Week 2 highlighted how process prevents miscommunication, reduces rework, and improves quality. A strong workflow is foundational for real-world engineering.

Week 3 — Software Design, OOP, UML

1. Introduction

Week 3 focused on software design as a mechanism for controlling complexity. Strong design leads to systems that scale and adapt without breaking.

2. OOP Principles

- **Abstraction** — exposes essential details only
- **Encapsulation** — protects internal state
- **Inheritance** — reuses behavior
- **Polymorphism** — flexible method behavior across types

These principles ensure cleaner, more organized code.

3. APIEC Model

The APIEC model combines:

1. Abstraction
2. Polymorphism
3. Inheritance
4. Encapsulation
5. Composition

APIEC helps avoid code duplication and supports modular architecture.

4. UML for Design

UML diagrams reduce ambiguity and guide implementation:

- **Use Case** diagrams describe interactions
- **Class** diagrams show structure
- **Sequence** diagrams model message flow

5. HW4 Application

UML modeling supported:

- Identifying core entities (Students, Courses, Users)
- Establishing controller–model relationships
- Designing authentication flow
- Reducing misinterpretations during implementation

6. Reflection

Week 3 emphasized that design is essential—not optional. High-quality software is planned before it is coded.

Week 4 — SOLID Principles, Testing, and Clean Architecture

1. Introduction

Week 4 expanded on design by introducing principles used to build scalable, maintainable, and testable systems.

2. SOLID Principles

- **S: Single Responsibility Principle**
- **O: Open/Closed Principle**
- **L: Liskov Substitution Principle**
- **I: Interface Segregation Principle**
- **D: Dependency Inversion Principle**

These principles reduce coupling and enhance flexibility.

3. Testing

Testing ensures stability and confidence:

- Unit testing
- Integration testing
- System testing
- Regression testing

Testing also prevents defects from spreading across the system.

4. Composition Over Inheritance

Composition is preferred because it avoids rigid hierarchies and encourages modular, evolvable systems.

5. HW4 Application

HW4 demonstrated SOLID and architectural best practices:

- SRP enforced through separate files
- DIP implemented via Eloquent ORM
- OCP through extendable endpoints
- Postman used for consistent test cycles

6. Reflection

Week 4 showed that disciplined architecture ensures long-term code quality and adaptability.

Week 5 — High-Level Programming with JavaScript

1. Introduction

Week 5 explored how JavaScript, a high-level language, reduces complexity and accelerates development through abstraction.

2. High-Level Abstraction

JavaScript eliminates many low-level concerns:

- Memory handling
- Manual data structures
- Networking details
- Complex threading models

This allows developers to focus on solving actual problems.

3. JavaScript for Web Applications

JavaScript is ideal for web systems because:

- It works natively with browsers
- JSON is its native data format
- Async features improve responsiveness
- NPM provides reusable packages

4. Dynamic Typing

JavaScript’s dynamic typing enables creativity but increases the risk of runtime errors, motivating the transition to TypeScript.

5. HW4 Application

JavaScript supported HW4 by:

- Processing JSON responses
- Testing API calls
- Debugging with console tools
- Rapid prototyping

6. Reflection

Week 5 demonstrated that high-level languages increase developer efficiency and reduce accidental complexity.

Week 6 — TypeScript and React

1. Introduction

Week 6 covered TypeScript and React, both essential for building modern, robust web applications.

2. TypeScript and Static Typing

TypeScript introduces:

- Compile-time type checking
- Explicit interfaces
- Safer refactoring
- Stronger code clarity

This significantly reduces runtime failures.

3. React and Component Architecture

React provides structure through:

- Reusable components
- One-way data flow
- Virtual DOM optimization
- Clear separation of UI and logic

4. TypeScript + React

Together they deliver:

- Safer components
- Explicit contracts
- Fewer undefined/null errors
- Scalable front-end architecture

5. HW4 Application

If applied to HW4:

- TypeScript would model Student, Course, User entities
- React would render forms, lists, dashboards
- Strong typing would catch data mismatches early

6. Reflection

Week 6 showed that modern tools make quality easier by enforcing structure and reducing uncertainty.

Overall Reflection

Across Weeks 1–6, quality software emerged as the result of:

- Managing complexity
- Following structured processes
- Designing before coding
- Applying SOLID principles
- Testing continuously
- Leveraging high-level tools

HW4 served as a practical application of these principles.

Conclusion

Quality software is intentional.
It results from thoughtful planning, strong communication, clean design, modern tooling, and disciplined engineering practices.
These lessons will guide future academic and professional development.